

TRADEGE PLATFORM

Core Trading Engine Algorithm - Step-by-Step Technical Guide

Document Overview

This document details the step-by-step lifecycle of Buying and Selling operations (Market Orders, Limit Orders, Stop Loss, and Take Profit checks) within the TradeEdge Spring Boot backend. For each phase, the corresponding Java code structure and logical operations are explained in a simplified, highly detailed format.

Table of Contents

- 1. Phase 1: Order Placement & Validation (Market & Pending Orders)**
- 2. Phase 2: Pending Order Execution (Background Evaluator Task)**
- 3. Phase 3: Risk Management (Automatic SL/TP Position Liquidator)**
- 4. Phase 4: Manual Position Closing & Balance Settlement**

TRADEGE PLATFORM

Core Trading Engine Algorithm - Step-by-Step Technical Guide

Phase 1: Order Placement & Validation

Source Location:

`com.brokerui.broker.BrokerApiController.java --> placeOrder(...)`

When a trader clicks BUY or SELL, the request containing the Symbol, Side, Order Type, Volume (Quantity), and optional SL/TP rates hits the API controller. The algorithm proceeds as follows:

Step 1.1: Validation: Checks parameters (e.g. quantity must be positive).

Step 1.2: Margin Reservation (for LIMIT/STOP orders): Verifies if account balance is greater than the required Margin (derived as $\text{targetPrice} * \text{volume} * \text{contractSize} / \text{leverage}$). If ok, it immediately deducts and reserves this margin from the balance, then saves the order with status "NEW".

Step 1.3: Margin Deduction (for MARKET orders): Immediately checks and deducts required margin at the current live price. A Position is opened (LONG side for BUY, SHORT side for SELL).

Step 1.4: Neural Network (AI) Routing: Invokes `applyNnRouting()` to analyze order size, spread, imbalance, and order book depth to recommend internal matching or MT5 bridge routing.

Key Code Implementation (Simplified):

TRADEGE PLATFORM

Core Trading Engine Algorithm - Step-by-Step Technical Guide

Phase 2: Pending Order Execution (LIMIT / STOP)

Source Location:

`com.brokerui.broker.OrderExecutionService.java --> tick() & tryExecute(...)`

Pending orders waiting in status "NEW" are evaluated by a scheduled background worker thread running every 2,000 milliseconds (2 seconds):

Step 2.1: Query: Fetches all orders currently saved with status "NEW" in the database.

Step 2.2: Price Check: Retrieves the live quote price of the symbol from Bybit (for Crypto) or Yahoo (for Forex/ Metals).

Step 2.3: Trigger Condition Check: Checks if the target trigger price has been reached by comparing the market price:

- BUY LIMIT: Triggers if `livePrice <= limitPrice`
 - SELL LIMIT: Triggers if `livePrice >= limitPrice`
 - BUY STOP: Triggers if `livePrice >= stopPrice`
 - SELL STOP: Triggers if `livePrice <= stopPrice`
- ```
Boolean shouldFill = switch (type) {
 case "BUY LIMIT" -> last.compareTo(limit) <= 0 : last.compareTo(limit) >= 0;
 case "SELL LIMIT" -> last.compareTo(limit) >= 0 : last.compareTo(limit) <= 0;
 case "BUY STOP" -> last.compareTo(stop) >= 0 : last.compareTo(stop) <= 0;
 case "SELL STOP" -> last.compareTo(stop) <= 0 : last.compareTo(stop) >= 0;
 default -> false;
```

**Step 2.4:** Execution (`executeFilled`): If the condition matches, the order is filled, a LONG/SHORT position is created, the order status changes to "FILLED", and the trade is routed to MT5 if recommended.

```
if (shouldFill) executeFilled(order, last);
```

#### Key Code Implementation (Simplified):

# TRADEGE PLATFORM

## Core Trading Engine Algorithm - Step-by-Step Technical Guide

### Phase 3: Risk Management (Automatic SL/TP)

#### Source Location:

`com.brokerui.broker.OrderExecutionService.java --> checkPositionsSLTp()`

To protect clients and manage risk, a scheduler running every 2 seconds inspects all open positions with Stop Loss (SL) and Take Profit (TP) parameters set:

**Step 3.1:** Query Active Positions: Scans all active rows in the database position table.

**Step 3.2:** Evaluate SL/TP Limits: Loads live price and compares it against target SL/TP values:

- Stop Loss Trigger: Triggers if a LONG position price falls  $\leq$  SL, or if a SHORT position price rises  $\geq$  SL.
- Take Profit Trigger: Triggers if a LONG position price rises  $\geq$  TP, or if a SHORT position price falls  $\leq$  TP.

**Step 3.3:** Auto-Close Position (Close Position Due to SL/TP): Instantly deletes the position, computes final P/L, returns margin + profit to balance, writes a filled closing order to history, and fires a push notification (if triggered/tp triggered).

```
closePositionDueToSLTp(pos.getId(), last, "STOP LOSS");
if ((tp != null && (!isShort && last.compareTo(tp) >= 0) || (isShort && last.compareTo(tp) <= 0))) {
```

**Key Code Implementation (Simplified):**

```
closePositionDueToSLTp(pos.getId(), last, "TAKE_PROFIT");
}
```

# TRADEGE PLATFORM

## Core Trading Engine Algorithm - Step-by-Step Technical Guide

### Phase 4: Manual Position Closing & Settlement

#### Source Location:

`com.brokerui.broker.BrokerApiController.java --> closePosition(...)`

When a trader manually clicks the close "" button in the open positions list, the endpoint processes the liquidation:

**Step 4.1:** Price Feed Acquisition: Pulls the final closing market price quote.

**Step 4.2:** Margin Refund: Calculates the initial locked margin:  $\text{avgPrice} * \text{quantity} * \text{contractSize} / \text{leverage}$ . This margin is set to be returned to the balance.

**Step 4.3:** P/L Realization: Computes profit or loss:

- LONG P/L =  $(\text{closePrice} - \text{avgPrice}) * \text{quantity} * \text{contractSize}$   
- SHORT P/L =  $(\text{avgPrice} - \text{closePrice}) * \text{quantity} * \text{contractSize}$

**Step 4.4:** Balance Adjustment: Adjusts the balance:  $\text{balance} \leftarrow \text{balance} + \text{marginReturned} + \text{P/L}$ .

**Step 4.5:** Clean up: Deletes the active Position row from database, and inserts a closing FILLED order transaction in history for auditing.

#### Key Code Implementation (Simplified):